

Basics — just enough to read the definitions

For **Q6** (speech synthesis), **Q8** (reinforcement learning) and **Q9** (animation), where the answers in [DEFINITIONS.md](#) use machinery that is never explained there.

How to use it: read the section for one topic, then immediately read that Q in [DEFINITIONS.md](#). It should now parse. Don't try to memorize anything here — this is scaffolding, and the exam tests the definitions, not this.

Shared vocabulary

Four words that appear in all three topics.

Embedding — a list of numbers standing in for something that isn't a number (a word, a speaker, an entity). Similar things get similar lists, so the model can do arithmetic on meaning.

Encoder / decoder — an **encoder** squeezes an input into a compact internal representation; a **decoder** expands that back out into an output. Text → (encoder) → internal → (decoder) → speech.

Autoregressive — generating a sequence **one piece at a time, each conditioned on the pieces already produced**. Accurate, because every step sees what came before, but slow, because you can't start step 100 until step 99 is done. The opposite is **parallel / non-autoregressive**: emit everything at once, fast but each piece is blind to the others.

Autoencoder — an encoder and decoder trained together to **reproduce their own input**, forced through a narrow middle called the **latent space**. Since everything must pass through that bottleneck, the network is compelled to keep only what matters. The latent space ends up holding the handful of variables that really describe the data.

NOTES

Q6 — Speech synthesis

The core idea: two stages, not one

Sound is a **waveform** — air pressure sampled thousands of times a second (22,050 numbers per second is typical). Predicting that directly from text is brutal: one second of speech is 22,050 outputs.

So TTS splits the job in two:

text → [acoustic model] → mel-spectrogram → [vocoder] → waveform

Once you see this split, most of Q6 falls into place — every model named in the answers belongs to one half or the other.

What a spectrogram is

Chop the audio into short frames (about 20 ms). For each frame, record **how much energy sits at each frequency**. Stack the frames side by side and you get a picture: time across, frequency up, brightness = energy. That's a **spectrogram**.

A **mel-spectrogram** is the same picture with the frequency axis spaced the way humans hear — fine detail low down, coarse up high (this is recipe 3 in `CALC_RECIPES.md`).

Why bother: a mel-spectrogram is about 80 numbers per frame at about 100 frames/second, versus 22,050 raw samples. Far easier to predict, and it throws away detail the ear doesn't use.

Phase. A spectrogram says how *much* of each frequency is present but not *where the wave peaks* — that timing information is the **phase**, and it's discarded. To rebuild real audio you need it back. That's the vocoder's problem, and it's why Griffin-Lim exists: it *guesses* phase by iterating.

The text side

Before any audio, the text has to be cleaned up:

1. **Text normalization** — turn written text into spoken words: Dr. → “doctor”, 2026 → “twenty twenty-six”. **Homographs** are the hard case: same spelling, different sound (“*live* animals” vs “I *live* here”), resolved by a dictionary plus part-of-speech tagging.
2. **Phonetic analysis (grapheme-to-phoneme)** — letters to **phonemes**, the distinct sound units of the language. A **phone** is one such sound; a **diphone** runs from the middle of one phone to the middle of the next, so it captures the *transition* between them.
3. **Prosody** — the melody of speech: which syllables are **stressed**, and the **intonation**, meaning the pitch contour over the sentence. Pitch is written F_0 , the fundamental frequency — how fast the vocal folds vibrate. Rising F_0 at the end signals a question.

Old approach: unit selection

Record a person for hours, cut the audio into diphones, store them. To say a new sentence, pick fragments from the database and stitch them together. Two costs decide which:

- **target cost** — does this fragment have the pitch, stress and duration I want?
- **join cost** — will it splice cleanly onto the next one? Zero if the two were already neighbours in the original recording, since then there's no seam at all.

Quality is limited by what's in the database. Neural TTS replaced it.

Modern acoustic models

Both produce a mel-spectrogram from phonemes; they differ in *how*:

- **Tacotron 2** is **autoregressive** — one spectrogram frame at a time, using **attention** to decide which phoneme it's currently pronouncing. Natural, but slow, and the attention can slip and skip or repeat a word.
- **FastSpeech** is **non-autoregressive** — all frames at once. But to emit everything in parallel it must know *in advance* how many frames each phoneme occupies, so it has a **duration predictor**. One phoneme becomes several spectrogram columns by repeating the encoder state that many times. Fast and robust. Scaling the durations directly controls speaking speed — **more repeats = longer sounds = slower speech**, which is the 2024 MC trap.

Vocoders (mel-spectrogram → waveform)

- **Griffin-Lim** — no learning, just iteratively estimates the missing phase. Fast, worst quality.
- **WaveNet** — predicts the waveform **one sample at a time** using **causal dilated convolutions** (“causal” = only looks backwards; “dilated” = skips gaps to see far back cheaply). Superb quality, painfully slow, since it’s autoregressive over 22,050 samples per second.
- **HiFi-GAN** — a **GAN**: a generator makes audio while a discriminator tries to spot fakes, and they train against each other. Produces the whole waveform in parallel, so it’s both fast and good — the real-time choice.

Style transfer

To control *who* is speaking, feed the model a **speaker embedding** alongside the text. Either one-hot labels (speaker #3 of 5), a learned embedding from a large corpus (which generalizes to new speakers — **zero-shot**), or an embedding extracted automatically from a reference clip’s spectrogram, which needs no labels at all.

→ Now read **Q6 in DEFINITIONS.md**.

NOTES

Q8 — Reinforcement learning

The setup

An **agent** interacts with an **environment** in a loop. At each timestep it sees the situation, does something, and gets a score:

state s_t $\rightarrow$ agent picks action a_t $\rightarrow$ environment returns reward r_t and a new state s_{t+1}

That’s the whole framework. The vocabulary:

- **State** — a complete description of the situation right now (position, velocity, goal location).
- **Action** — what the agent can do (move left, apply torque to a joint).
- **Policy** $\pi(a | s)$ — the agent’s brain: the rule mapping state to action. **This is the thing being learned.** Everything else is scaffolding.
- **Reward** r_t — a single number scoring that one step.
- **Return** — the **total** reward accumulated over a whole run. This is what you actually want to maximize; a single reward is myopic.
- **Trajectory** — one complete run from start to finish, also called an **episode** or **rollout**. It’s the *data* of RL: you generate it by letting the agent loose and recording what happened.
- **Markov Decision Process (MDP)** — the formal name for this setup. “Markov” means the next state depends **only on the current state and action**, not on the whole history.

The goal: find the policy that maximizes expected return.

Why not just use supervised learning?

Two reasons, both worth knowing because the exam asks:

1. **There are no labels.** Nobody can tell you the single correct joint torque for a walking robot. You only find out whether a *whole sequence* of actions worked out.
2. **The environment isn’t differentiable.** Normal neural network training works by **backpropagation**: nudge each weight in the direction that reduces the error, which requires knowing how the output changes as you wiggle each input — a derivative. A physics simulator is just code; there’s no derivative running through it. So you can’t backpropagate from “the robot fell over” back to the network weights.

Consequence, and a favourite MC trap: **the reward does not need to be differentiable**. It can be any number you like, including something discontinuous like “+1 if you scored”.

Stochastic policies

Rather than outputting one action, the policy usually outputs a **probability distribution** over actions, and you sample from it. For continuous actions, the network emits the **mean and variance of a Gaussian**; for a discrete set, a **softmax** giving a probability per action. Randomness matters because it makes the agent **explore** instead of repeating one mediocre habit forever.

How learning actually happens

Policy gradient, the basic idea: run some episodes, then increase the probability of actions that preceded a high return and decrease those that preceded a low one. Simple, and it works.

Its problem — stated in the slides — is **high variance**: returns swing wildly between episodes depending on luck, so the updates are noisy and training is unstable and can diverge.

The fix is a baseline. Instead of asking “was the return high?”, ask “was it **higher than usual for this state?**” To do that you need an estimate of the usual, which is a **value function**: the expected return from a given state. Subtracting it cancels most of the noise.

That gives **actor-critic (A2C)**:

- **actor** = the policy, choosing actions
- **critic** = the value function, judging how good the situation was

PPO adds one more guardrail: don’t let the policy change too much in a single update. Big jumps based on noisy data wreck a policy that was working, so PPO clamps how far it can move. Stable, which is why it’s the default in practice.

On-policy vs off-policy

On-policy methods (policy gradient, A2C, PPO) can only learn from data generated by the **current** policy — the moment you update it, your old trajectories are stale and get thrown away. Wasteful, but simple and stable.

Off-policy methods (Q-learning, DDPG, SAC) can reuse old data from a **replay buffer**, so they need far fewer interactions with the environment.

Q-learning learns $Q(s, a)$: the expected return of taking action a in state s . Once you have it there's no separate policy to store — just take the best-scoring action, $a^* = \arg \max_a Q(s, a)$. That's what “the policy is implicit” means. (*Slides mark this “for self-study” — low priority.*)

→ Now read **Q8** in **DEFINITIONS.md**.

NOTES

Q9 — Animation

How a character is built

Two layers:

- A **mesh** — the visible surface, thousands of points (**vertices**) joined into triangles.
- A **skeleton** — a hierarchy of **bones** connected at **joints**, hidden inside.

You animate the skeleton, and the mesh follows. The link between them is **skinning**: each vertex is assigned **weights** saying how much each nearby bone influences it (a vertex on the elbow is pulled partly by the upper arm, partly by the forearm).

Linear Blend Skinning is the simple rule for this: move the vertex to the weighted average of where each bone would put it,

$$v'_i = \sum_{j=1}^B w_{ij} (R_j v_i + T_j)$$

with R_j the bone’s rotation and T_j its translation. Cheap and it’s what games use — but averaging *rotations* linearly is not really valid, and at **large joint angles or twisting** the surface pinches and loses volume. The slides call this the “**bow tie**” effect.

Posing: FK and IK

- **Forward kinematics** — you set the joint angles, and compute where the hand ends up. Easy: just chain the bones together.
- **Inverse kinematics** — you say where the hand must *go*, and solve for the angles that put it there. Much harder, and much more useful (“grab that ball”, “keep the foot on the floor”).

The **end-effector** is the tip of the chain — the hand, the foot — the bit you’re aiming.

There’s no clean formula for IK, so you solve it by nudging: work out which way to rotate each joint to reduce the distance to the target, take a small step, repeat. The **Jacobian** is what tells you how the hand moves in response to each joint rotating. That’s recipe 13 in `CALC_RECIPES.md`, and doing that calculation once will teach you this better than any paragraph.

Keyframes and the Disney principles

Traditional animation is **pose to pose**: the animator draws the important poses at chosen frames (**keyframes**), and the computer fills in the frames between by **interpolating**.

Interpolating in a straight line at constant speed looks wrong, because nothing in the physical world starts and stops instantly. Hence **Slow In and Slow Out**: real objects **accelerate and decelerate**, so motion should be **non-linear**. The 12 Disney principles are a set of such rules of thumb for making motion read as alive.

Data-driven animation

Modern work learns from **motion capture** — recordings of real people moving.

The difficulty is dimensionality. A single pose is about 60 numbers (a joint angle per degree of freedom), so a motion is a path through 60-dimensional space. But almost every point in that space is a grotesque, impossible body. **Real human motion occupies a tiny, curved sliver of it** — and that sliver is called a **manifold**.

Manifold learning finds the handful of variables that actually describe the data (speed, gait, direction) and maps the big space down to a small one, $f : \mathbb{R}^d \rightarrow \mathbb{R}^m$ with $m < d$. In practice this is an **autoencoder**: the latent space *is* the manifold. Working in those few variables makes generating and blending motion tractable.

DeepPhase builds on this. Walking is **periodic** — the same cycle repeating — so it uses a **periodic autoencoder** to extract a **phase** variable saying *where in the walk cycle* you are. Line two motions up by phase and they blend cleanly instead of the feet sliding along the ground.

Normalizing flows solve a different problem: you want a model that can both **generate** new motion and **say how likely** a given motion is. The trick is to build the transformation out of steps that are each **invertible** — runnable forwards and backwards. Start from a simple distribution (a Gaussian, the **prior**), push it through the chain to get complex realistic motion; run the chain backwards to compute the exact probability of a real motion. Most generative models can do one or the other; flows do both.

→ **Now read Q9 in DEFINITIONS.md.**

NOTES

If you only remember three things

1. **Q6:** TTS is text $\rightarrow$ mel-spectrogram $\rightarrow$ waveform. Acoustic models do the first arrow, vocoders the second. Every named model sits in one of those two boxes.
2. **Q8:** everything exists to learn the **policy**. Rewards judge steps, returns judge runs, value functions reduce noise, PPO stops the policy lurching.
3. **Q9:** you animate a **skeleton** and the **mesh** follows. FK is angles $\rightarrow$ position, IK is position $\rightarrow$ angles, and the modern methods all lean on motion living on a low-dimensional **manifold**.

NOTES
